

INDIVIDUAL ASSIGNMENT 2

ML Pipeline Design & MLOps Analysis

Telecom Customer Churn

A reproducible retention-risk system

Trịnh Đức Dương

Student ID: 25ms13290

Course: DDM501 - AI in DevOps, DataOps, MLOps

Submission: Individual written report

Revised: 6 October 2026

Companion GitHub repository

<https://github.com/TrinhDucDuong/ddm501-assignment-2-telco-churn-mlops-pipeline>

Implementation snapshot: [627989c](#). The PDF is self-contained; code, data and execution evidence supplement the report.

Contents

Executive brief and continuity	3
1. Pipeline architecture and data flow	4
2. Data contract, features and scaling rationale	5
3. Experiment design and tracking strategy	6
4. Metrics strategy and measured results	7
5. MLflow implementation excerpts	9
6. Workflow orchestration design	10
7. Code quality, tests and configuration	11
8. Reproducibility and versioning	12
9. Operational assessment and next production gates	13
10. Submission evidence and reproduction	13
References and rubric map	14

Executive brief and continuity

This assignment extends the telecom retention system designed in Assignment 1. The problem is to rank customers by churn risk so a retention team can allocate a limited contact budget. The original architecture separated validated snapshots, train-only transformations, model evaluation, controlled serving and feedback. This submission turns those boundaries into executable stages and evaluates ten configurations with MLflow tracking and an Airflow DAG.

The scenario, public IBM fictional sample, nine predictors, random seed 42 and 60/20/20 split remain unchanged. Assignment 1's Logistic Regression C=1 model becomes experiment E04, allowing a directly traceable comparison. The new capabilities are a checksum gate, persisted stage outputs, systematic validation comparisons, a model registry, a guarded local release and scored customer export. Both projects contain independent copies of their required code and data and are published as separate Git repositories. The [Assignment 1 design report](#) is the baseline design referenced here.

The distinction between demonstration and deployment remains important. The dataset has no temporal snapshots, the labels do not prove prospective validity, and no real retention intervention is observed. The business goal remains positive incremental retained margin under a randomized pilot. This pipeline demonstrates reproducible offline model selection and local MLOps execution; it does not claim realized revenue or production availability.

Continuity item	Assignment 1	Assignment 2 extension
Business decision	Capped retention review queue	Same decision; top-20% scored export
Model development	Prior and logistic baseline	Ten controlled configurations
Evaluation	Fixed train/validation/test partitions	Same partitions; validation-only selection
Model storage	Local pipeline artifact	MLflow runs, model versions and candidate alias
Workflow	Python CLI	Same stage functions called by Airflow
Evidence	Tests and baseline metrics	Tests, experiment table, registry and DAG evidence

1. Pipeline architecture and data flow

IMPLEMENTED PIPELINE | persisted boundaries and shared stage functions

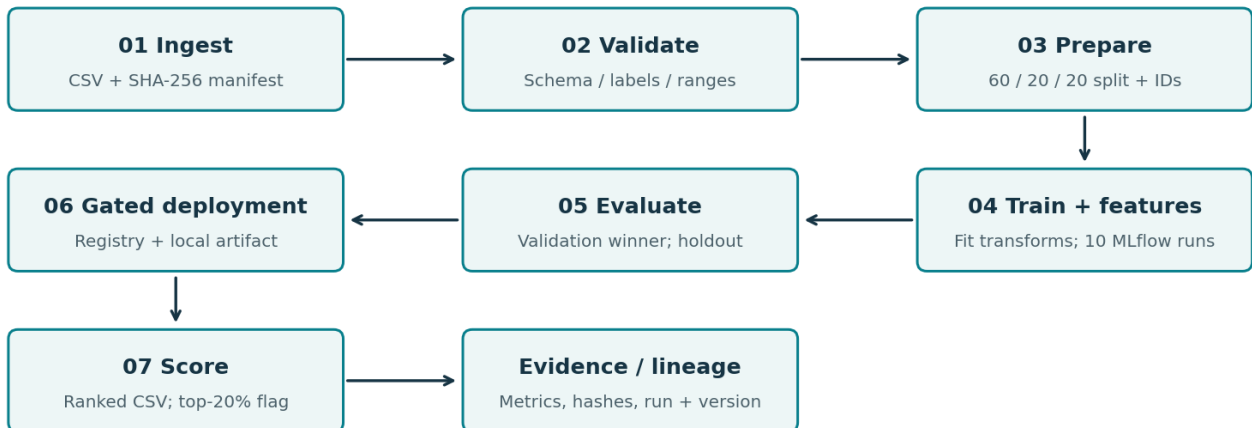


Figure 1. Implemented pipeline and persisted data transformations.

The pipeline has seven persisted stages: ingest, validate, prepare, train, evaluate, deploy and score. Preparation defines the split and input features; learned preprocessing and feature encoding execute within each training trial. This avoids fitting a global transformer before partitioning. One serialized scikit-learn Pipeline carries the imputer, scaler, encoder and estimator into inference.

1.1 Stage specifications

Stage	Input to output	Operations and quality gate
Ingest	Source CSV and manifest to raw.csv	Compute SHA-256, compare manifest, copy immutable snapshot; mismatch fails
Validate	raw.csv to validated.csv and validation.json	Required fields, unique ID, label vocabulary, categories, finite nonnegative numbers
Prepare	Validated snapshot to train/validation/test CSVs and split_ids.json	Seeded stratification; 60/20/20 split before fitting; no overlapping IDs
Preprocess / features (inside train)	Training fields to encoded matrices	Training median, standard scaling, one-hot encoding; exclude ID and label
Train	Train/validation and matrix to trial artifacts, experiments.csv, winner.json	Fit ten pipelines; log parameters, validation metrics, signatures and hashes
Evaluate	Selected candidate and holdout to summary.json	Evaluate only preselected winner; record AP, AUC, Brier, threshold and capacity metrics
Deploy	Summary and candidate to registry version and local model.joblib	Validation AP ≥ 0.60 , lift ≥ 2.0 and AP gain over prior ≥ 0.05
Score	Approved local artifact and simulated incoming cohort to scored_customers.csv	Validate features, finite probabilities, stable descending rank, capped contact flag

The score stage uses the holdout as a simulated incoming batch and does not feed its labels into prediction. This provides reproducible end-to-end evidence without pretending the sample is a live CRM feed. A real daily scoring job must consume the latest unlabeled eligible cohort and refresh consent/contact rules independently of weekly model training.

1.2 Modularity and recovery

Each stage accepts the same configuration mapping and persists a named output. The CLI can execute a whole run or a single stage; Airflow calls exactly the same stage functions. Small XCom return values are artifact paths, not feature matrices or trained model objects. This keeps orchestration metadata small and separates model logic from the scheduler.

Raw input is never modified. Validation failure stops downstream processing. A failed training trial is visible as a failed MLflow run; Airflow retries may repeat training and create additional experiment runs, which is acceptable for this small demonstration and is documented rather than described as exactly-once execution. Deployment checks for an existing version with the same MLflow run ID so a task retry reuses that version. Local model publication uses a temporary file and atomic file replacement; the registry alias and local artifact are separate state changes and are not a distributed transaction.

On a failed quality gate the deployed model bytes and champion manifest are preserved, which is covered by a regression test. A failure between registry registration and local publication can leave an unused candidate version; retry reconciles by run ID. Production should use a single authoritative immutable release manifest, approvals and rollout reconciliation before promoting an alias to production.

2. Data contract, features and scaling rationale

The bundled source is the IBM Telco Customer Churn sample: 7,043 rows and 1,869 positive labels. The data manifest records the original URL and full SHA-256. This snapshot is kept independently in each project. No network access is required to train once dependencies have been installed.

The selected numeric features are tenure, MonthlyCharges and TotalCharges. Categorical predictors are Contract, InternetService, PaymentMethod, PaperlessBilling, OnlineSecurity and TechSupport. customerID is used only to trace records; Churn is the label. Demographics and remaining source columns are dropped. Feature selection is fixed across the current experiment matrix to isolate algorithm and hyperparameter effects. Feature-set and preprocessing ablations are proposed follow-up experiments, not additional configurations falsely counted as executed trials.

TotalCharges contains eleven blanks, all at zero tenure. They remain missing until the per-trial training median imputer fits. Scaling statistics and category vocabulary also come only from training rows. The strict external schema rejects unknown categories, while the internal encoder uses `handle_unknown='ignore'` as a defensive serialization boundary. Missing categories and invalid labels are rejected, not converted to arbitrary defaults.

The fixed partitions contain 4,225 training rows, 1,409 validation rows and 1,409 holdout rows. Customer ID sets are stored and tested for disjointness and repeatability. No global oversampling or scaling occurs before splitting. Imbalance is addressed by ranking metrics and one explicitly balanced logistic trial rather than artificially rebalancing the whole dataset.

2.1 Scalability decisions

Sequential trials are adequate for this dataset and avoid nested parallelism. Random forests explicitly use one estimator-level worker. At production size, parallel trial tasks can share a read-only, versioned snapshot and write separate run directories. An object store should replace local artifact paths, and a PostgreSQL-backed MLflow service should replace SQLite for concurrent writers. Airflow workers must see the same artifact store; the local SequentialExecutor does not demonstrate multi-node access or high availability.

Scaling the data volume changes more than CPU demand: point-in-time joins, schema evolution, retention policy and label maturation become first-class contracts. A temporal holdout is required for live deployment. The current random split is useful for a controlled educational comparison but cannot estimate performance under seasonality, pricing changes or new acquisition channels.

3. Experiment design and tracking strategy

The prior-probability classifier is the noninformative baseline. Logistic C=1 from Assignment 1 is the reference engineering baseline. All trials use identical rows, selected features, preprocessing, capacity fraction and random seed. The complete matrix is encoded in config.yaml, so the result table can be regenerated without manual notebook state.

ID	Algorithm	Hyperparameters varied	Purpose
E01	Dummy prior	Training class prevalence	Noninformative ranking baseline
E02	Logistic	C=0.01, unweighted	Strong regularization
E03	Logistic	C=0.1, unweighted	Moderate regularization
E04	Logistic	C=1, unweighted	Assignment 1 reference
E05	Logistic	C=10, unweighted	Weaker regularization
E06	Logistic	C=1, class_weight=balanced	Recall and calibration trade-off
E07	Random forest	100 trees, depth=6, min leaf=5	Restrained nonlinear interactions
E08	Random forest	200 trees, depth=12, min leaf=5	Higher capacity and latency
E09	Gradient boosting	100 trees, depth=2, learning_rate=0.05	Slow additive learning
E10	Gradient boosting	150 trees, depth=2, learning_rate=0.1	More aggressive boosting

Logistic regression uses max_iter=2000. Tree models receive the same seed, and random forests use n_jobs=1. No feature extraction uses target aggregates. Subsequent experiments could compare service-only versus service-plus-billing features, standard versus robust scaling, or calibrated probability estimates. Those changes need a new predeclared matrix and appropriate validation; they are not implied by the current results.

3.1 Tracking schema and version relationships

One MLflow experiment groups the trials. Each run logs algorithm, name, hyperparameters, seed, capacity and feature list; metrics include validation AP, ROC-AUC, Brier, F1, precision/recall at 0.5, precision/recall/lift at 20%, training duration and warm single-row model p95 latency. Tags include data SHA-256, Python-source SHA-256 and partition identity. Artifacts include the split IDs and complete serialized preprocessing/model pipeline with input example and signature.

The selected run subsequently receives test-prefixed metrics and the summary artifact. Its registry version is linked to that exact run. The candidate alias is movable; the numeric version and run ID are immutable references for audit. champion.json records the local demonstration's deployed version and artifact checksum. Candidate here is intentionally not a production approval claim.

4. Metrics strategy and measured results

Primary selection uses validation average precision (AP), with exact ties broken by lower fit duration then stable experiment name. AP is the stepwise precision-recall summary implemented by scikit-learn, not an arbitrary trapezoidal PR-area calculation. The selection function ignores any test metric. Held-out predictions are computed for the chosen model only; the baseline holdout metrics in Assignment 1 remain historical context.

Secondary measures answer different questions. ROC-AUC measures overall ordering across thresholds but may conceal poor positive-class precision. Precision@20% estimates how many contacted customers belong to the labeled risk group under the contact budget. Recall@20% measures coverage of that group. Lift normalizes queue precision by prevalence. Brier assesses probability error, while F1 at 0.5 exposes the balance of false positives and false negatives for a diagnostic threshold. No metric measures causal response to a retention offer.

All ten rows below are measured on the same validation partition. Timing is hardware- and load-dependent.

Trial	AP	AUC	Brier	F1	Lift@20%	Fit (s)	p95 (ms)
E01	0.2654	0.5000	0.1950	0.0000	1.042	0.011	3.71
E02	0.6684	0.8526	0.1345	0.5789	2.592	0.016	5.65
E03	0.6726	0.8553	0.1328	0.6006	2.592	0.023	3.95
E04	0.6737	0.8563	0.1324	0.6156	2.578	0.036	4.90
E05	0.6746	0.8565	0.1323	0.6187	2.592	0.032	5.15
E06	0.6731	0.8559	0.1660	0.6355	2.578	0.023	5.61
E07	0.6731	0.8578	0.1320	0.5692	2.605	0.313	12.21
E08	0.6725	0.8538	0.1322	0.5688	2.498	0.726	16.45
E09	0.6725	0.8557	0.1324	0.5732	2.538	0.429	3.80
E10	0.6687	0.8528	0.1326	0.5881	2.498	0.592	5.65

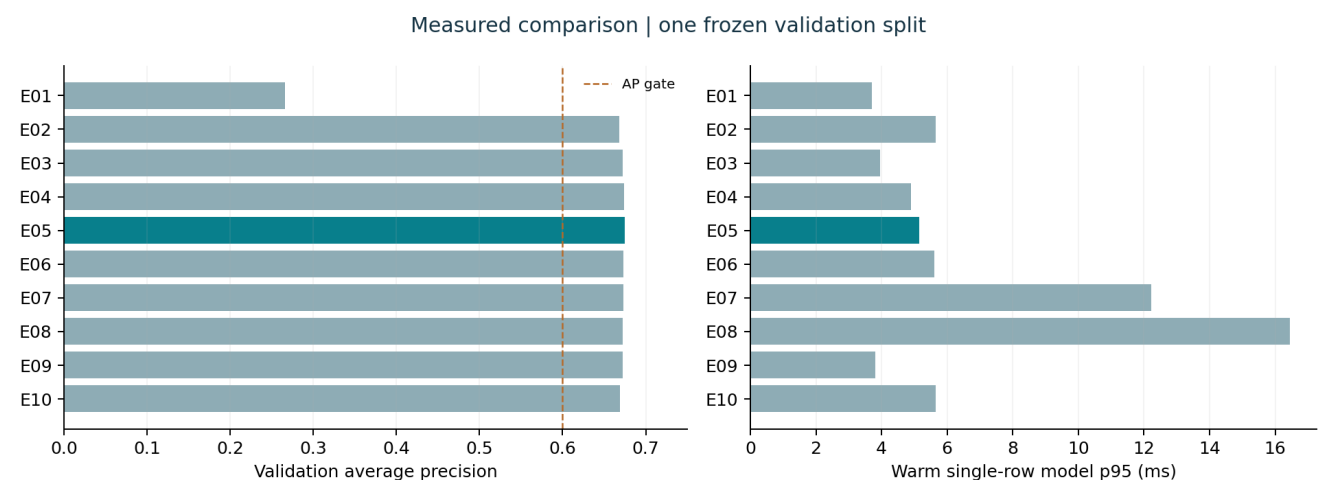


Figure 2. Measured AP and latency; timing varies with host workload.

Selected candidate: E05-lr-c10. Validation AP=0.6746; validation lift@20%=2.5917. The quality gate passes. No model was reselected based on the following holdout table.

Holdout metric	Measured value
average precision	0.6250
roc auc	0.8296
brier	0.1425
f1	0.5669
precision at capacity	0.6348
recall at capacity	0.4786
lift at capacity	2.3914

Registry version: 3; model: ddm501-telco-churn; alias: candidate. The run ID and full lineage are preserved in evidence/champion.json and the local MLflow store.

The holdout contains 1,409 customers. The 20% queue contains 282 records. At threshold 0.5, the confusion counts are TN=911, FP=124, FN=177, TP=197. Threshold classifications and the top-budget queue are different policies.

Execution evidence: Python 3.10.11; project-local environment; dependency, lint, test and full-data pipeline commands exited successfully. A real HTTP server returned 200 for valid prediction and 422 for an invalid empty batch. Machine-readable evidence and full test output are in evidence/verification.json and evidence/pytest.txt. Execution timestamp (UTC): 2026-10-03T16:48:09.345259+00:00.

Actual Airflow execution: success; 7 of 7 tasks completed; 1409 scored rows and 282 contact flags were persisted from the Docker run. See evidence/airflow.txt and evidence/airflow-summary.json. This validates real DAG execution, not continuous scheduling or HA.

4.1 Analysis and recommendation

E05 is the validation-AP winner, but its advantage over E04 is approximately 0.00084 AP. This very small difference is not evidence of a statistically meaningful improvement. It is appropriate to register E05 as the mechanically selected local candidate while retaining E04 as a plausible production alternative pending repeated temporal validation and operational review.

Increasing regularization from $C=10$ to $C=0.01$ slightly reduces AP and changes threshold recall. Class weighting in E06 raises recall at threshold 0.5 substantially, but lowers precision and worsens Brier score. Its top-budget lift is nearly unchanged, illustrating why threshold metrics and ranking metrics must be interpreted separately. A larger F1 is insufficient reason to replace the AP winner when a fixed contact budget drives the application.

The shallow random forest has competitive AP and slightly stronger top-budget precision/ROC-AUC in this split, but higher inference cost. The deeper forest does not improve validation AP despite greater capacity and runtime. More aggressive boosting likewise does not beat the simpler logistic model. These patterns suggest that the fixed features contain strong approximately additive signals, although a single split cannot establish this as a general property of telecom churn.

The recommendation is a logistic pipeline as the initial candidate family: low training cost, compact artifact, straightforward interpretation and competitive ranking. E05 is selected under the predeclared rule; performance uncertainty, calibration and real intervention economics determine whether it should be deployed beyond the demonstration. Never choose an algorithm by looking at all test outcomes and then label the winning test result unbiased.

5. MLflow implementation excerpts

The executable implementation is `churn/mlops.py`. These excerpts show the actual API choices; the full file includes validation, artifact paths, retry handling and metadata. SQLite is suitable for this single-user exercise. The tracking URI can be overridden through `MLFLOW_TRACKING_URI` to use a managed tracking server without rewriting training functions.

```
mlflow.set_tracking_uri(uri)
client = MlflowClient()
if client.get_experiment_by_name(name) is None:
    client.create_experiment(
        name, artifact_location=(out / "mlruns").as_uri()
    )
mlflow.set_experiment(name)

with mlflow.start_run(run_name=spec["name"]) as active:
    model = build_model(spec, config["seed"])
    model.fit(train, train.Churn.eq("Yes").astype(int))
    mlflow.log_params(**spec, "seed": config["seed"])
    mlflow.log_metrics(metrics)
    mlflow.set_tags({"data_sha256": data_hash,
                    "source_sha256": source_hash()})
    mlflow.log_artifact(str(out / "split_ids.json"))
    mlflow.sklearn.log_model(
        model, "model",
        signature=infer_signature(example, model.predict(example)),
        input_example=example,
        pip_requirements=["scikit-learn==1.7.2",
                        "pandas==2.3.3", "numpy==2.2.6"],
    )
```

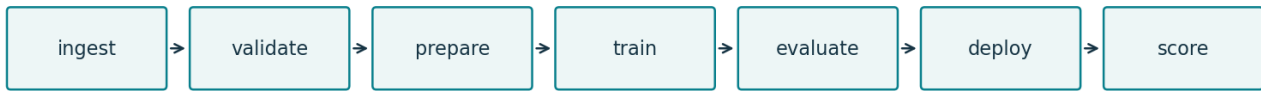
The logged scikit-learn model's generic predict signature returns class labels; ranking code deliberately calls the loaded sklearn pipeline's `predict_proba` method. A production probability-only MLflow serving interface would wrap this method in a custom pyfunc contract rather than assume generic predict returns probabilities.

```
check_gate(summary, config["gates"])
version = mlflow.register_model(f"runs:{run_id}/model", name)
client.set_registered_model_alias(name, "candidate", version.version)
```

The full deploy function first searches versions by run ID, so retries do not blindly create a new version. It stages and reloads the local artifact before publication. Registration provides traceability, not authorization to contact customers. Production promotion requires owner approval, a fresh cohort evaluation, consent controls and a canary/rollback plan.

6. Workflow orchestration design

AIRFLOW DAG | telco_churn_training



Sunday 02:00 Asia/Ho_Chi_Minh | all_success | 2 retries | max_active_runs=1

Figure 3. Executable Airflow task dependencies and schedule.

The training DAG is a linear dependency chain because each stage consumes persisted evidence from its predecessor. Every task uses the default all-success trigger rule, so validation or gate failure prevents deployment and scoring. At this small scale, preprocessing and all experiment fits share one train task to keep recovery simple; a larger design can fan out ten trial tasks and join them at winner selection without changing the data contract.

The planned automated trigger is weekly at 02:00 Sunday in Asia/Ho_Chi_Minh. `start_date` is fixed, `catchup=False` prevents unintended historical retraining, `max_active_runs=1` bounds concurrent releases, and a 30-minute DAG timeout limits runaway executions. Manual triggers support data fixes or approved drift investigations. Daily inference is a separate production cadence, not a claim that this weekly training DAG alone produces fresh daily CRM queues.

```

with DAG(
    dag_id="telco_churn_training",
    start_date=pendulum.datetime(2026, 1, 1,
                                tz="Asia/Ho_Chi_Minh"),
    schedule="0 2 * * 0", catchup=False, max_active_runs=1,
    dagrun_timeout=timedelta(minutes=30),
    default_args={"retries": 2,
                  "retry_delay": timedelta(seconds=15),
                  "retry_exponential_backoff": True,
                  "execution_timeout": timedelta(minutes=15),
                  "on_failure_callback": report_failure},
) as dag:
    tasks = [PythonOperator(
        task_id=name, python_callable=execute_stage,
        op_kwargs={"stage_name": name}
    ) for name in STAGES]
    for upstream, downstream in zip(tasks, tasks[1:]):
        upstream >> downstream
  
```

6.1 Run isolation and operational considerations

The task wrapper hashes Airflow `run_id` to obtain a safe run-directory key. All tasks and retries for that run use the same directory; another run gets another directory. No raw feature data is put into XCom. Each run has its own local MLflow store in the demonstration; production should use a centralized registry with all runs pointing to immutable object-store prefixes.

Two retries with exponential backoff address transient read/write failures, while schema and quality errors still stop the pipeline after retries. `report_failure` writes a structured event containing DAG, task and run ID. This is implemented logging; delivery to Slack, email or an incident service is a production integration requirement, not demonstrated notification. Monitor task duration, retry count, input freshness, missingness, registry failures and completed score count. Page an operator for missed freshness deadlines or repeated source failures; send model-quality alerts to the model owner after labels mature.

The reproducible Docker command uses Airflow's real dags test path. It executes the DAG and its task code in process with the Airflow metadata database; it is not a mock DAG test. It does not prove a

continuously running scheduler, webserver, multi-worker executor or high-availability deployment. The evidence log and persisted scored output distinguish actual task execution from merely importing a DAG file.

7. Code quality, tests and configuration

The project separates data contracts, model construction, metric calculation, baseline pipeline, MLOps stages, HTTP serving and orchestration. Public functions include concise docstrings and type annotations; comments explain nonobvious choices such as stable tie sorting, retry-safe registration and artifact publication. Ruff checks common Python errors and formatting. Tests exercise behavior rather than checking source strings.

Data tests reject duplicate IDs, invalid categories/labels, negative or infinite numeric values and established customers with missing charges. Split tests assert determinism and disjoint membership. A preprocessing test verifies that prediction on extreme inputs does not alter training statistics. A hand-computed metric fixture verifies capacity selection and lift. Real integration tests train, serialize, reload and serve a small model; API tests cover valid and invalid HTTP payloads. Extended tests protect validation-only winner selection, gate rejection, artifact preservation, checksum failure and repeat registration.

```
seed: 42
capacity: 0.2
data_path: data/telco.csv
output_dir: artifacts
experiment_name: ddm501-telco-churn
model_name: ddm501-telco-churn
gates:
  min_average_precision: 0.60
  min_lift: 2.0
  min_ap_gain: 0.05

config = yaml.safe_load(path.read_text(encoding="utf-8"))
if not 0 < config.get("capacity", 0) <= 1:
    raise ValueError("capacity must be in (0,1]")
value = Path(os.environ.get("CHURN_DATA_PATH", config["data_path"]))
config["data_path"] = str(value if value.is_absolute()
                           else path.parent / value)
```

CHURN_DATA_PATH and CHURN_OUTPUT_DIR override configured paths; MLFLOW_TRACKING_URI overrides tracking; CHURN_MODEL_PATH selects an API artifact; CHURN_CONFIG selects the Airflow configuration. Relative paths resolve against the config file rather than an unrelated working directory. Credentials are not stored in YAML. Each project owns its .venv, requirements, source, data, tests and Docker configuration; local environments and generated runtime databases are excluded from Git.

8. Reproducibility and versioning

8.1 Code, data and model lineage

The project is published in its own [GitHub repository](#) on main. The [implementation snapshot 627989c](#) preserves the code and evidence underlying this report. The existing initial commit was retained. A checked-in GitHub Actions workflow defines dependency installation, lint, tests and pipeline verification; successful local checks do not by themselves establish a successful hosted CI run.

For future development, use short-lived feature branches, reviewed pull requests and a protected main branch. Branch protection and release tags are proposed governance controls, not claimed existing repository settings. Link each approved release to a Git commit, data hash, configuration and registry version. From a cloned repository, the following illustrates a future feature and release workflow; v1.0.0 has not been published as part of this submission.

```
git switch -c feature/new-temporal-cohort
git add config.yaml churn tests
git commit -m "Evaluate a new temporal cohort"
# After review, tests and merge:
git tag -a v1.0.0 -m "Validated churn pipeline release"
```

The source CSV is small enough to version directly for this exercise; source.json captures its hash and provenance. Large private datasets should instead be versioned in object storage with immutable manifests or DVC pointers, never committed as raw customer records. Record snapshot time, label definition, extraction query version, schema and retention policy. Git tracks code and configuration, the data manifest identifies exact bytes, and MLflow maps the experiment run to a numeric model version.

```
train, remaining = train_test_split(
    frame, test_size=0.4, stratify=frame.Churn, random_state=42
)
model = LogisticRegression(max_iter=2000, random_state=42, C=10)
forest = RandomForestClassifier(random_state=42, n_jobs=1)
```

All stochastic splitting and estimators receive an explicit seed. Dependency versions are pinned, and each environment records a complete freeze for the executed platform. Seeds do not guarantee byte-identical models across operating systems, BLAS libraries or Python versions; compare stable metrics with tolerances and preserve the original artifact when exact replay matters. Runtime IDs, timestamps and measured latencies naturally differ between runs.

8.2 Container contract and artifact portability

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY churn churn
COPY config.yaml .
COPY data data
CMD ["python", "-m", "churn"]
```

The dedicated Airflow image starts from apache/airflow:2.10.2, installs the ML dependencies and copies this project's DAG and modules. The Linux image and Windows virtual environment are separate tested execution paths; a Windows environment directory is never copied into Docker. Pin an image digest and archive a platform-specific lock for stricter supply-chain reproducibility. Existing local MLflow artifact URIs contain absolute paths. A clean Git checkout excludes runtime artifacts and creates a fresh store; moving a folder with its old database requires a new empty CHURN_OUTPUT_DIR. The pipeline detects a mismatched experiment artifact location and explains this recovery procedure. Published CSV/JSON evidence remains readable without the original machine.

9. Operational assessment and next production gates

The design supports auditable offline development: data is fingerprinted, transforms are fitted on training only, selection uses validation, the registered candidate is traceable, and deployment is blocked by explicit gates. It also makes failures observable through exit codes, persisted reports and Airflow task state. Those are meaningful MLOps controls at the scope of a local individual assignment.

The next production gates are a dated dataset with a 30-day forward label, chronological evaluation, subgroup and calibration analysis, measured load/cost at intended volume, authenticated inference, consent-aware CRM integration and randomized business evaluation. A centralized registry and immutable release manifest should coordinate canary deployment and rollback. Drift is an investigation signal, not permission to replace a model automatically.

The experiment winner's apparent improvement over Assignment 1 is tiny, and the holdout was already visible in the earlier assignment. This is a continuity limitation, not a fresh independent replication. No additional tuning was based on holdout outcomes. A genuinely unseen future cohort is needed to estimate generalization after model selection and business-policy design.

10. Submission evidence and reproduction

The submitted written report is DDM501_Assignment2_25ms13290_TrinhDucDuong.pdf. The pipeline diagram, ten-configuration experiment matrix and measured results, DAG, MLflow snippets, configuration and versioning examples are included in the PDF itself. The repository supplements the written analysis with complete executable files.

Repository: [ddm501-assignment-2-telco-churn-mlops-pipeline](https://github.com/25ms13290/ddm501-assignment-2-telco-churn-mlops-pipeline). Clone it independently; Assignment 1 is a conceptual predecessor and is not a runtime dependency. From the repository root on Windows with Python 3.10 or 3.11:

```
powershell -ExecutionPolicy Bypass -File setup.ps1
./venv/Scripts/python.exe scripts/verify.py
```

This runs dependency checks, lint, tests, the full ten-trial pipeline and a real HTTP smoke test. Inspect generated artifacts/experiments.csv, summary.json and champion.json for experiment, holdout and model lineage. With Docker Desktop using Linux containers, execute the orchestration separately:

```
docker compose build airflow-test
docker compose run --rm airflow-test
```

Completed check	Recorded result	Published evidence
Unit and integration tests	23 passed; no failures	Test output
Real HTTP and registry loading	Health OK; valid 200; invalid 422; registered model loaded	Verification record
Controlled experiment matrix	Ten configurations; E05 selected on validation AP	Measured experiment CSV
Real Airflow DAG execution	7/7 tasks successful; 1,409 scores; 282 contact flags	Airflow summary

These are recorded completed runs. New run IDs, durations and registry versions may differ. The report revision updates documentation and repository links without rerunning model selection or changing the conclusions. Runtime environments, generated models and MLflow databases are rebuilt locally and excluded from Git; the published evidence remains readable without rebuilding them.

References and rubric map

- [1] IBM. Telco Customer Churn sample. <https://github.com/IBM/telco-customer-churn-on-icp4d>
- [2] scikit-learn. Common pitfalls and recommended practices. https://scikit-learn.org/stable/common_pitfalls.html
- [3] MLflow 2.22.0. Tracking and Model Registry. <https://mlflow.org/docs/2.22.0/tracking.html> and <https://mlflow.org/docs/2.22.0/model-registry.html>
- [4] Apache Airflow 2.10.2. DAGs, testing and best practices. <https://airflow.apache.org/docs/apache-airflow/2.10.2/core-concepts/dags.html> and <https://airflow.apache.org/docs/apache-airflow/2.10.2/best-practices.html>
- [5] scikit-learn. Average precision score. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html

Rubric	Report sections and executable evidence
Pipeline design (25%)	Sections 1-2; pipeline diagram; churn/mlops.py
Tracking and metrics (25%)	Sections 3-5; ten-row results; artifacts/experiments.csv
Orchestration (20%)	Section 6; DAG diagram; dags/churn_training.py; Airflow log
Code and documentation (20%)	Section 7; tests; config; README; type hints and docstrings
Reproducibility (10%)	Section 8; manifest; pinned requirements; Dockerfiles; registry

Author: Trịnh Đức Dương. Student ID: 25ms13290. All numerical experiment results inserted into this report are sourced from executed artifacts; business benefits and production SLOs are explicitly proposed rather than observed.